Watermark Removal: Literature Review and Analysis of Three Attacker Access Levels

The strongest approach for your Part C is to treat removal as three different information settings: blind rewriting, detector-guided rewriting, and key-guided rewriting. The literature supports examining all three [1]–[6], [9], [10], but it does **not** justify claiming that removal always succeeds.

I mapped the research to your uploaded repository and assignment. The attack outcomes below are **predictions, not experiments we have completed**.

First, what counts as removal in our project?

Your detector calculates:

It flags text when . Therefore, successful removal requires:

- An initially flagged **and usable** watermarked passage.
- An edited score **at or below 0.60**.
- Preserved meaning and readability, checked against both the watermarked passage and the original.

Your colors are fixed by **the key and normalized word**. Reordering unchanged words does not change their colors or the score. This differs from the required paper's context-dependent generation watermark. [7]

What the literature contributes

Research	Relevant finding	Implication for our project
Krishna et al., NeurIPS 2023	Paragraph-level paraphrasing can evade	Supports blind paraphrasing as a removal strategy. It does

— DIPPER [1]	watermark detectors while approximately preserving semantics.	not establish our success rate. proceedings.neurips.cc
Kirchenbauer et al., ICLR 2024 — On the Reliability of Watermarks [2]	Paraphrasing can leave detectable watermark evidence, especially when enough text remains available.	We should not write “paraphrasing removes the watermark” as a guarantee. Their statistical detector also differs from our fixed-fraction threshold. proceedings.iclr.cc
Zhao et al., ICLR 2024 — Provable Robust Watermarking [3]	A fixed-group, unigram watermark can have editing robustness under its theoretical assumptions.	Fixed colors are not automatically a bad design. However, their guarantees do not transfer directly to our WordNet substitutions and detector. proceedings.iclr.cc
Jovanović et al., ICML 2024 — Watermark Stealing [4]	Learning watermark preferences through a watermarked generation API improves removal and spoofing.	Supports studying information leakage. Their access is to a **generation API**, whereas our query cell concerns a **detector**; these are different capabilities. proceedings.mlr.press
Zhang et al., ICML 2024 — Watermarks in the Sand [5]	Repeated quality-preserving perturbations can erase watermarks under assumptions about reliable quality assessment and efficient exploration of alternative texts.	Motivates iterative rewriting without knowing the key. The result is conditional, not proof that every practical attack succeeds. proceedings.mlr.press
Harel-Canada	Practical random-walk	Strong justification for your

et al., ACL 2025 — Sandcastles in the Storm [6]	attacks suffered from slow progress and imperfect quality assessment. Reported success fell from 26% to 10% under human quality review in their experiments.	assignment's independent quality check: a lower score alone is insufficient. ACL Anthology
Wu & Chandrasekaran, ACL 2024 — Color-Aware Substitutions [9]	A "color-aware" attack infers which tokens are green and replaces them with non-green ones, evading detection with fewer edits than color-blind attacks.	Closest published match to key-access removal. They must infer colors by prompting the model; a key holder reads them directly. Their setting is a generation-time watermark, so their results do not transfer to our scheme.
Shi et al., TACL 2024 — Red Teaming Language Model Detectors [10]	A language model proposes context-appropriate synonyms; substitutions are applied within an edit budget, optionally selected with detector scores. Detection falls while the text stays plausible.	Supports using a context-aware generator instead of WordNet to preserve meaning. With the key, the selection can run locally without queries. Their detection numbers do not transfer to our scheme.

Cell 1: Removal with no detector access

The attacker knows the rules but cannot query the detector or calculate colors.

The best-supported strategy is **meaning-preserving lexical paraphrasing**: replace wording, reconstruct sentences, and retain facts, names, relationships, and qualifications. Paraphrasing can weaken watermark

detection [1], although residual evidence may remain detectable [2]. Repeated quality-preserving rewriting can erase watermarks under strong assumptions [5], but practical attacks were slower and succeeded less often under human quality review [6].

For our implementation, changing vocabulary matters more than simply changing word order. A rewrite that preserves exactly the same word counts produces exactly the same score.

Expected costs and limitations:

- **Detector queries:** zero.
- **Editing effort:** potentially substantial, because the attacker cannot identify helpful substitutions or know when to stop.
- **Expected effect:** sufficiently different wording may weaken the original green bias, but particular edits can also increase the score. Kirchenbauer et al. make the same point: without knowing the watermark, each changed token has only a 50% chance of falling in the red list [7].
- **Quality risk:** extensive rewriting may change details, tone, or meaning.

A useful stopping condition is a predetermined editing budget followed by an independent assessment. The evaluator may measure the final score afterward; that information must not guide the blind attacker.

What could prevent success? Too few meaning-preserving alternatives, too little rewriting, or retention of many green-word occurrences. Rate limits do not defend against an attacker who makes no queries.

Added techniques for Cell 1 (predictions)

Technique A: **blind look-alike letters**. The attacker knows the word rules, which count any Unicode letter as part of a word. Replacing letters in random word occurrences with look-alikes (for example a Cyrillic “e” for a Latin “e”) makes each edited word a new word with an unknown, effectively random color, while the text looks the same to a reader.

- Pick random word occurrences, replace some of their letters with look-alikes, and continue until about half or more of the words are changed. Without detector access the attacker cannot see the score, so they must edit well past the minimum.

- **Expected cost and limits (Technique A):** zero queries. In a simulation with our key and the 10 watermarked outputs (300 trials per setting), editing 40% of the words ended at or below 0.60 in 82% of trials and editing 60% in 96%; reaching a 90% chance took 25% to 70% of the words (about 45% typically). Quality risk: the text looks identical when displayed, but copying, searching and screen readers reveal the change. Defense: normalize look-alike letters and remove invisible characters before tokenizing (this changes the frozen word rules and needs a separate evaluation), and watermark at a target well above the threshold.

Technique B: **automated paraphrasing**. The attacker passes the passage through a paraphrasing model or a back-translation round trip. This is the automated form of the lexical paraphrasing described above, and it needs no detector access and no key.

- **Expected cost and limits (Technique B):** zero queries and little editing time. A paraphraser keeps many function words, so we predict it changes roughly 30% to 60% of the tokens, close to the share we estimate is needed (about 9% to 50%, median 38%). Success is therefore plausible but uncertain, and some watermark evidence may remain [2]. Quality risk: the meaning may change, and the blind attacker cannot check the result. Defense: a watermark target well above the threshold, so more rewriting is needed.
- Weaker variant, zero-width splitting: inserting an invisible character inside a word splits it into two new words. In the same simulation it was much less reliable (about 37% success at 40% of the words and 55% at 60%, and E-W-05 stayed flagged even at 60%), because splitting produces few distinct fragments whose fixed colors depend on the key. We therefore treat it as a weaker option.

- **Support and attribution:** Look-alike and zero-width attacks, and normalizing text as the defense, are discussed in [7] and [8]. Automated paraphrasing is supported by [1]; [2] shows that residual watermark evidence can survive paraphrasing, and [7] also discusses paraphrasing attacks. The simulation results, the cost estimates, and the 30% to 60% paraphrasing estimate are our own analysis of this project’s code and data, not results from these papers. All numbers are predictions, not measurements.

Cell 2: Removal with detector-query access

The attacker can test chosen inputs but cannot read the key.

The primary strategy is **detector-guided substitution**:

1. Propose a contextually appropriate replacement.
2. Query the permitted detector interface.
3. Use the response to select further edits.
4. Stop within the budget when an acceptable passage becomes unflagged.

With a numeric score, the attacker can observe gradual progress. With only a binary answer, two flagged candidates do not reveal which is closer to passing.

However, **our implementation has an additional, more direct leakage possibility**:

> If the service accepts an isolated word, its score is either 0 or 1. Even a binary decision at threshold 0.60 reveals that word's color.

If the service merely requires more words but permits repetition, repeating the same word produces the same disclosure. This follows from our code; it is **not a measured result or a finding copied from the papers**. Related work on learning watermark rules queries a watermarked generation API, not a detector [4].

Consequently, "binary responses prevent color learning" would be an incorrect claim unless input restrictions actually block these probes.

Expected costs and limitations:

- For straightforward candidate testing: approximately **one baseline query plus one query per candidate tested**.
- For direct word probing: **one accepted query per distinct word tested**, plus passage verification.
- Editing effort remains necessary because a red synonym can still be semantically wrong.
- Query counts depend on input restrictions, response precision, and rate limits.

Defenses: enforce query budgets, limit informative responses, and examine restrictions on short or highly repetitive inputs. Binary responses alone do not

resolve the leakage described above. Any restriction also needs testing against legitimate use.

Added techniques for Cell 2 (predictions)

Technique A: **query-guided look-alike letters**. The tokenizer counts any Unicode letter as part of a word, so a Cyrillic “e” in place of a Latin “e” turns “the” into a different word that looks identical to a reader. The new word gets a new, unknown color, which the attacker can check with the detector.

- Probe the most frequent words of the passage with repeated-word texts and keep those that are green.
- For each green word, probe a look-alike version. About half will be red, so roughly two probes per word are expected.
- Replace the green occurrences with the red look-alikes until the detector reports the passage unflagged or the query budget ends.
- Why it may help: function words such as “the”, “I” and “a” supply many of the green occurrences (they are the cheapest swaps in our outputs) but have no safe synonyms. Look-alikes keep the meaning and the visible text. A zero-width character inside a word works in the same way: it splits “the” into “t” and “he”, and the attacker probes both pieces to find a split in which both are red.

- **Expected cost and limits (Technique A):** about 30 to 60 queries per passage (conservative estimate: probing the frequent words plus about two probes per swapped word) and 2 to 13 changed occurrences. Quality risk: the text looks identical when displayed, but copying, searching and screen readers reveal the change. Defense: normalize look-alike letters and remove invisible characters before tokenizing. This changes the frozen word rules, so it would need a separate evaluation.

Technique B: **dilution by insertion**. Instead of replacing words, the attacker inserts words that probing has shown to be red, such as “indeed” or “moreover”, so the green share falls. The score is at most 0.60 when $G / (N + m) \leq 0.60$, so the attacker needs $m \geq G / 0.60 - N$ inserted words (G green occurrences, N total words).

- **Expected cost and limits (Technique B):** for our 10 watermarked outputs this is 3 to 21 inserted words (2% to 17% extra text), plus about two probes per red filler word found. Quality risk: the original

words and meaning stay, but too much filler harms the style. Defense: limit query budgets so red fillers cannot be found, and watermark at a target well above the threshold so more insertions are needed.

- **Support and attribution:** Look-alike and zero-width attacks, and normalizing text as the defense, are discussed in [7] and [8]. Insertion attacks are discussed in [7], and [3] counts insertions among the edits its robustness bound allows. The combination with detector probing, the cost estimates, and the insertion formula are our own analysis of this project's code, not results from these papers. All numbers are predictions, not measurements.

Cell 3: Removal with key access

The attacker can calculate word colors locally.

The natural strategy is **targeted green-to-red substitution**: locate green occurrences, generate suitable alternatives, and choose red replacements that preserve meaning and grammar. Wu and Chandrasekaran [9] use the same idea.

For a fixed-length passage with words and green occurrences, successful one-for-one green-to-red changes must satisfy:

$$k \geq \lceil G - 0.60 \times N \rceil$$

For example, a hypothetical 100-word passage with 70 green occurrences needs **10 successful green-to-red occurrence changes** to reach 0.60. This assumes each replacement remains one normalized word and no other colors change. Zhao et al. rely on the same worst case: each edit can turn at most one green token red [3].

Expected costs and limitations:

- **Remote queries:** zero needed for local color calculations; one final service check may verify the outcome.
- **Editing effort:** search for appropriate red alternatives and review the resulting prose.
- **Main limitation:** knowing colors does not create suitable synonyms. Some essential words cannot be replaced safely.

Defense: prevent key disclosure. Once the attacker knows the key, detector rate limits cannot stop offline color calculations. Rotating keys has consequences for verifying previously watermarked text, so it is not a cost-free solution.

Added techniques for Cell 3 (predictions)

Support for the core strategy: **color-aware substitution**. Wu and Chandrasekaran [9] describe the same idea, replacing green tokens with non-green ones, and report that it needs fewer edits than color-blind attacks. They must infer the colors; a key holder computes them directly, so the attacker's only remaining cost is finding suitable replacements.

- **Project numbers (computed with our key):** for the 10 watermarked outputs, 2 to 13 green occurrences (1 to 3 different words) are enough to reach 0.60 or below. The cheapest words are function words such as "the", "I", "a" and "that", which cannot be replaced safely, so the attack has to use content words. Each output has 32 to 71 green content-word occurrences (words of four or more letters, excluding common function words) against the 2 to 13 swaps needed. The swaps are therefore possible numerically, but each one needs a red replacement that keeps the meaning. Each edit removes at most one green word, which is the worst case bounded in [3].

Technique A: **context-aware candidate generation with a key filter**. Instead of a WordNet lookup, which changed the meaning of all 10 outputs, the attacker asks a language model for replacements that fit the sentence, as in Shi et al. [10], and keeps only the candidates that the key marks as red. Shi et al. apply such substitutions within an edit budget; with the key, the attacker can also recompute the score locally after each swap and stop at 0.60, so no detector query is needed.

- **Expected cost and limits (Technique A):** zero detector queries; the effort is reviewing each swap. About half of all candidates are red, so a few candidates per word are enough. Quality risk: lower than with WordNet because the replacement is chosen for its context, but any swap can still change a nuance. Defense: keep the key secret.

Technique B: **key-checked look-alike letters**. The attacker replaces a letter in a green word with a look-alike (for example a Cyrillic "e") and keeps the

variant only if the key marks it red. The text looks identical when displayed. Look-alike and invisible-character attacks are discussed in [7] and [8].

- **Expected cost and limits (Technique B):** zero queries; one or two variants per word are enough, and 2 to 13 occurrences suffice for our outputs. Unlike synonyms, it also works on function words. Quality risk: invisible when displayed, but copying, searching and screen readers reveal it. Defense: normalize look-alike letters and remove invisible characters before tokenizing; this changes the frozen word rules and needs a separate evaluation.

- **Context dependence:** in Kirchenbauer et al. [7] a changed token also changes the next token's green list, so one edit can create up to two violations. Our colors depend on the word alone, so each swap changes exactly one word's color, which is why the bound $k = \lceil G - 0.60 \times N \rceil$ holds for one-word replacements.

- **Support and attribution:** The core idea is supported by [9], context-aware substitution by [10], the per-edit bound by [3] (see also [7] for the roughly one quarter of the tokens that an informed attacker must modify against the hard watermark), and look-alike and invisible-character attacks by [7] and [8]. The project numbers, the content-word analysis, and the combination of key filtering with these techniques are our own analysis of this project's code and data, not results from these papers. All numbers are predictions, not measurements.

Two project-specific findings should shape our write-up

First, character changes need separate treatment. Your tokenizer lowercases text, so case changes alone cannot help. Punctuation between already separated words generally changes nothing; punctuation or zero-width characters **inside** words can split them and alter scoring. Whether that passes the quality criteria must be assessed rather than assumed. Kirchenbauer et al. also discuss zero-width and homoglyph attacks and recommend normalizing text before testing [7].

Second, the recorded evaluation_quality_review.csv marks **all ten outputs as failing meaning preservation**. Under the assignment's eligibility rule, there are currently **zero eligible removal inputs** in that reviewed set.

That does not prevent completing the three-cell threat analysis for the first submission. It means we must present these strategies as predictions and cannot report a valid removal success rate from those ten outputs. If eligibility remains zero for the later experiment, the assignment calls for **N/A and a clearly labeled artificial example**, rather than counting unusable passages as successful attacks.

References

[1] K. Krishna, Y. Song, M. Karpinska, J. Wieting, and M. Iyyer, “Paraphrasing evades detectors of AI-generated text, but retrieval is an effective defense,” in Advances in Neural Information Processing Systems, vol. 36, 2023. https://proceedings.neurips.cc/paper_files/paper/2023/hash/575c450013d0e99e4b0ecf82bd1afaa4-Abstract-Conference.html

[2] J. Kirchenbauer, J. Geiping, Y. Wen, M. Shu, K. Saifullah, K. Kong, K. Fernando, A. Saha, M. Goldblum, and T. Goldstein, “On the reliability of watermarks for large language models,” in International Conference on Learning Representations (ICLR), 2024. https://proceedings.iclr.cc/paper_files/paper/2024/hash/d78e9e4316e1714fbb0f20be66f8044c-Abstract-Conference.html

[3] X. Zhao, P. Ananth, L. Li, and Y.-X. Wang, “Provable robust watermarking for AI-generated text,” in International Conference on Learning Representations (ICLR), 2024. https://proceedings.iclr.cc/paper_files/paper/2024/hash/beae9ed5316bcc48e616754c06c11875-Abstract-Conference.html

[4] N. Jovanović, R. Staab, and M. Vechev, “Watermark stealing in large language models,” in Proceedings of the 41st International Conference on Machine Learning, PMLR, vol. 235, pp. 22570–22593, 2024. https://proceedings.mlr.press/v235/jovanovic24a.html

[5] H. Zhang, B. L. Edelman, D. Francati, D. Venturi, G. Ateniese, and B. Barak, “Watermarks in the sand: Impossibility of strong watermarking for language models,” in Proceedings of the 41st International Conference on Machine Learning, PMLR, vol. 235, pp. 58851–58880, 2024. https://proceedings.mlr.press/v235/zhang24o.html

[6] F. Y. Harel-Canada, B. Erol, C. Choi, J. Liu, G. J. Song, N. Peng, and A. Sahai, “Sandcastles in the storm: Revisiting the (im)possibility of strong watermarking,” in Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pp. 29698–29735, 2025, doi: 10.18653/v1/2025.acl-long.1436. https://aclanthology.org/2025.acl-long.1436/

[7] J. Kirchenbauer, J. Geiping, Y. Wen, J. Katz, I. Miers, and T. Goldstein, “A watermark for large language models,” in Proceedings of the 40th International Conference on Machine Learning, PMLR, vol. 202, pp. 17061–17084, 2023. https://proceedings.mlr.press/v202/kirchenbauer23a.html

[8] N. Boucher, I. Shumailov, R. Anderson, and N. Papernot, “Bad characters: Imperceptible NLP attacks,” in 43rd IEEE Symposium on Security and Privacy (S&P), 2022. https://arxiv.org/abs/2106.09898

[9] Q. Wu and V. Chandrasekaran, “Bypassing LLM watermarks with color-aware substitutions,” in Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pp. 8549–8581, 2024, doi: 10.18653/v1/2024.acl-long.464. https://aclanthology.org/2024.acl-long.464

[10] Z. Shi, Y. Wang, F. Yin, X. Chen, K.-W. Chang, and C.-J. Hsieh, “Red teaming language model detectors with language models,” Transactions of the Association for Computational Linguistics, vol. 12, pp. 174–189, 2024, doi: 10.1162/tacl_a_00639. https://aclanthology.org/2024.tacl-1.10